

EXCEPTION HANDLING IN JAVA

. EXCEPTION HANDLING IN JAVA

1. Title

Exception Handling – Checking Whether a Given Mobile Number Is Valid or Invalid Using Built-in and User-Defined Exceptions.

2. Aim

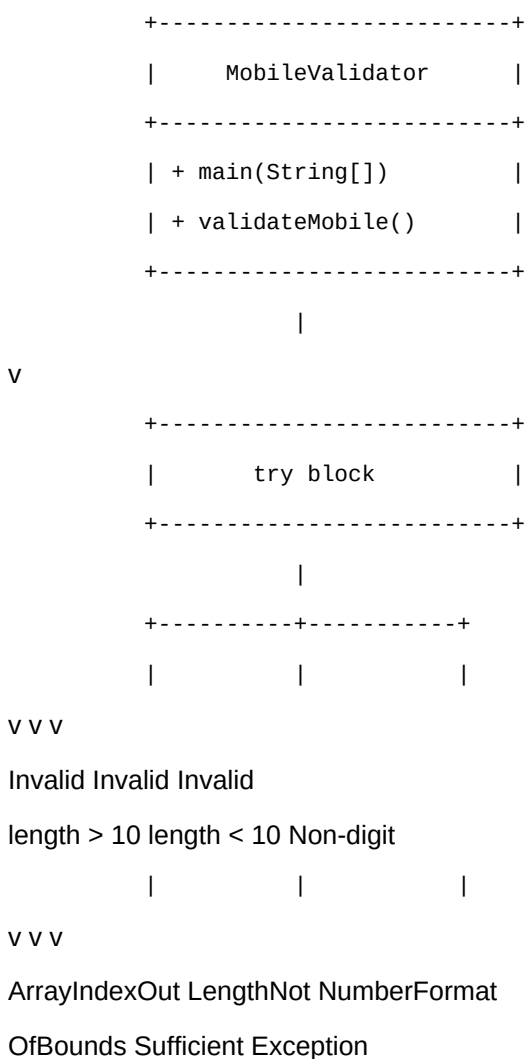
To write a Java program to input a mobile number and check whether it is valid or invalid using exception handling techniques such as "try", "catch", "throw", "throws", and user-defined exceptions.

3. Software and Hardware Requirements

Software: Java JDK and Visual Studio Code / Eclipse IDE.

Hardware: Computer or laptop with a minimum of 4 GB RAM.

4. UML Diagram



Exception Exception

```
      |           |           |
+-----+-----+
      |
```

v

```
+-----+
|   Display Result   |
+-----+
```

5. Algorithm

1. Start the program.
2. Read the mobile number as a string.
3. Check whether the input contains only digits.
4. If any character is not a digit, throw "NumberFormatException".
5. If the length exceeds 10 digits, throw "ArrayIndexOutOfBoundsException".
6. If the length is less than 10 digits, throw the user-defined "LengthNotSufficientException".
7. If the number contains exactly 10 digits, display "Valid number".
8. Handle the exceptions using multiple "catch" blocks.
9. Stop the program.

6. Source Code

```
import java.util.Scanner;
// User-defined exception
class LengthNotSufficientException extends Exception {
    public LengthNotSufficientException(String message) {
        super(message);
    }
}
public class MobileValidator {
    static void validateMobile(String number)
        throws LengthNotSufficientException {
        // Check whether all characters are digits
        for (int i = 0; i < number.length(); i++) {
            if (!Character.isDigit(number.charAt(i))) {
                throw new NumberFormatException();
            }
        }
        // Check whether the number exceeds 10 digits
        if (number.length() > 10) {
            throw new ArrayIndexOutOfBoundsException();
        }
        // Check whether the number has fewer than 10 digits
        if (number.length() < 10) {
            throw new LengthNotSufficientException(
                "Invalid Mobile Number - LengthNotSufficientException");
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter mobile number: ");
        String number = sc.nextLine();
        try {
```

```

        validateMobile(number);
        System.out.println("Valid number");
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println(
            "Invalid Mobile Number-ArrayIndexOutOfBoundsException");
    } catch (LengthNotSufficientException e) {
        System.out.println(e.getMessage());
    } catch (NumberFormatException e) {
        System.out.println(
            "Invalid Mobile Number -NumberFormatException");
    } finally {
        sc.close();
    }
}
}
}

```

7. Compilation and Execution Commands

```
javac MobileValidator.java
```

```
java MobileValidator
```

8. Output Screenshot

```

PS C:\Users\anits\Desktop\Java-Projects> javac MobileValidator.java
PS C:\Users\anits\Desktop\Java-Projects> java MobileValidator
Enter mobile number: 9853089465
Valid number

PS C:\Users\anits\Desktop\Java-Projects> java MobileValidator
Enter mobile number: 98567890121
Invalid Mobile Number-ArrayIndexOutOfBoundsException

PS C:\Users\anits\Desktop\Java-Projects> java MobileValidator
Enter mobile number: 98664433
Invalid Mobile Number - LengthNotSufficientException

PS C:\Users\anits\Desktop\Java-Projects> java MobileValidator
Enter mobile number: 98abc123
Invalid Mobile Number -NumberFormatException
PS C:\Users\anits\Desktop\Java-Projects>

```

Result

Thus, the Java program was successfully implemented to validate a mobile number using exception handling and display the appropriate message for valid and invalid inputs.